

Data Structures with C

Lecture 3: Dynamic Memory Allocation

Dr. Siddhant Thapliyal

August 1, 2026

Why Dynamic Memory Allocation?

Static memory allocation

- Memory size is fixed during compilation.
- May waste memory.
- Cannot grow during execution.

Dynamic memory allocation

- Memory is allocated at run time.
- Memory size depends on user requirements.
- Efficient utilization of memory.
- Allocation occurs in the Heap.

Static vs Dynamic Memory

Static Allocation

- Compile-time allocation
- Fixed size
- Stored in Stack/Data segment

Static vs Dynamic Memory

Static Allocation

- Compile-time allocation
- Fixed size
- Stored in Stack/Data segment

Dynamic Allocation

- Run-time allocation
- Flexible size
- Stored in Heap
- Managed using pointers

Typical Process Memory Layout

Conceptual Memory Organization

Higher Memory Address
Stack ↓ Local variables, function calls
Free / Unallocated Space
Heap ↑ Dynamic memory: <code>malloc()</code> , <code>calloc()</code>
BSS Segment Uninitialized global/static variables
Data Segment Initialized global/static variables
Text / Code Segment Program instructions
Lower Memory Address

Dynamic Memory Functions

Function	Purpose
<code>malloc()</code>	Allocates memory
<code>calloc()</code>	Allocates and initializes memory to zero
<code>realloc()</code>	Changes previously allocated memory size
<code>free()</code>	Releases allocated memory

malloc()

Syntax

```
ptr = (type *) malloc(size);
```

Example

```
int *arr;  
  
arr = (int *) malloc(10*sizeof(int));
```

Properties

- Allocates a block of memory.
- Contents are uninitialized.
- Returns NULL if allocation fails.

calloc()

Syntax

```
ptr = (type *)calloc(n, size);
```

Example

```
int *arr;  
  
arr = (int *)calloc(10, sizeof(int));
```

Difference from malloc()

- Allocates multiple blocks.
- Initializes all bytes to zero.

realloc()

Syntax

```
ptr = realloc(ptr, new_size);
```

Example

```
arr = realloc(arr, 20 * sizeof(int));
```

Used when

- Increase array size
- Decrease array size
- Preserve previous data whenever possible

free()

Syntax

```
free(ptr);
```

Example

```
free(arr);  
arr = NULL;
```

Advantages

- Prevents memory leaks.
- Returns memory back to Heap.
- Good programming practice.

Checking NULL Pointer

```
1 int *arr;  
2  
3 arr = (int *)malloc(10*sizeof(int));  
4  
5 if(arr==NULL)  
6 {  
7     printf("Memory Allocation Failed");  
8     exit(1);  
9 }
```

Always verify whether memory allocation is successful.

malloc() vs calloc()

malloc()	calloc()
Single block	Multiple blocks
Garbage values	Zero initialized
Faster	Slightly slower
One argument	Two arguments

Example Program

```
int n;

scanf("%d",&n);

int *arr =
(int *)malloc(n*sizeof(int));

for(int i=0;i<n;i++)
    scanf("%d",&arr[i]);

free(arr);
```

Common Mistakes

- Forgetting to call `free()`
- Not checking NULL pointer
- Using memory after `free()`
- Memory leaks
- Double `free()`
- Incorrect size calculation

Time Complexity

Operation	Complexity
malloc()	$O(1)$ (average)
calloc()	$O(n)$ (initialization)
realloc()	$O(n)$ worst case
free()	$O(1)$

Practice Questions

- 1 Differentiate between static and dynamic memory allocation.
- 2 Compare malloc() and calloc().
- 3 Explain realloc() with an example.
- 4 What is a memory leak?
- 5 Write a program to dynamically allocate an array of size n .

Questions?
Thank You